

Exam 2 Schedule:

Mon, Oct 27, 2025, 3:30pm-4:45pm
Classroom South 608

General Policies:

- The exam is closed book. Laptops/cellphones/headphones etc. are not allowed during the exam.
- You can bring one A4-size (copy paper size) **handwritten** double-sided note for the exam. PRINTED NOTES WILL NOT BE ALLOWED.
- Exam2 is 20% of the total grade.
- Exam 2's format will be similar to Exam1's format.
- Makeup exams will be considered only for strong reasons. Prior approval is required.

Syllabus:**1. Sorting Algorithms**

- Be able to **implement** and **describe the time complexity** in the best/worst case of the following:
 - Selection sort
 - Insertion sort
 - Merge sort
- Be able to identify and correct minor bugs in the implementation of the above.

2. Stacks, Queues, and Linked Lists

- Be able to implement the following in Python:
 - Stack (implementation using Python List)
 - Queue (implementation using Python List)
 - LinkedList
- Differences between Python Lists and Linked Lists.
- Be able to implement the following in code:
 - Inserting a node after 'kth' node of a linked list
 - Deleting the kth node from a linked list

3. Trees

Know tree terminology:

- Root, Leaf
- Child/Descendants, Parent/Ancestors
- Subtree
- Height
- Balance

- Be able to implement tree traversal algorithms (pre-order, in-order, and post-order).
- Be able to write pre/in/post-order traversal results for a given tree.
- Be able to write simple recursive tree functions.

4. Exceptions, Modules, File I/O

- Exceptions
 - Be able to identify the behavior of code snippets using try/except/finally
 - Be able to recognize common exception types (e.g. ZeroDivisionError, ValueError etc.)
 - Be able to define and raise custom exceptions.
- Modules
 - Be able to explain Python behavior around importing files.
 - import *, import foo as bar
 - from x import y ← still runs the entirety of x
 - Be able to explain what a module is and how it is helpful?
- File I/O
 - Be able to explain what the various open() modes are (e.g. 'r', 'w', 'a', 'r+')
 - Be able to choose the appropriate one for a given task.
 - Be able to recognize and explain the difference between relative and absolute paths.
 - Be able to use the core file API (read(), readlines(), write())
 - Write a Python function that reads a text file and prints the **word frequency**.
 - Write a Python function that reads a text file and prints the **frequency** of a given word.
 - Write a Python function that reads a text file and prints the **unique words**.
 - Write a Python function that **searches** for a word/string within a text file and return it's index (-1 if not found).

5. Graphs

Terminology

- Directed/Undirected, Weighted/unweighted, Acyclic/Cyclic.
- Edges, Cycles, Neighbors
- Be able to convert graph visuals into adjacency list and adjacency matrix representations, and vice-versa.
- Know how Breadth-First Search (BFS) and Depth-First Search (DFS) works .
(BFS/DFS implementation/coding is not required for this exam)
- Tree vs. Graph: differences